Blockchain Based Supply Chain Product Tracking System using Smart Contracts

1st Silvanus Alvan
line 2: *Faculty of Artificial inteligence and Data Science*
line 3: *Informatics*
line 4: Tangerang, Indonesia
line line 5: 01082230030@student.uph.edu

2nd Bisma Harun
line 2: *Faculty of Artificial inteligence and Data Science*
line 3: *Informatics*
line 4: Tangerang, Indonesia
line line 5: 01082230026@student.uph.edu

line 1: 3rd Hyun Lee
line 2: *Faculty of Artificial inteligence and Data Science*
line 3: *Informatics*
line 4: Tangerang, Indonesia
line line 5: 010822300xx@student.uph.edu

line 1: 4th Kent Richi Korompis
l line 2: *Faculty of Artificial inteligence and Data Science*
line 3: *Informatics*
line 4: Tangerang, Indonesia
line line 5: 01082230028@student.uph.edu

***Abstract*—Modern supply chains are characterized by complex, multi-party networks that generate substantial transaction data across geographically dispersed participants. Traditional centralized ledger systems face persistent challenges including data tampering, lack of transparency, trust deficits between parties, and audit inefficiency. This paper proposes a blockchain-based supply chain product tracking system that leverages smart contracts deployed on the Ethereum network to provide transparent, tamper-resistant, and fully traceable product lifecycle management. The system implements three core smart contract functionscreateProduct(), updateProduct(), and getProduct() with getProductHistoryItem() enabling authorized participants to interact with a shared immutable ledger via on-chain role-based access control (RBAC). The proposed architecture is analyzed against established blockchain security frameworks and compared with traditional centralized approaches. Results of the security and transparency analysis demonstrate that the system mitigates common supply chain fraud vectors including data manipulation, unauthorized record alteration, and audit inconsistency through blockchain properties of immutability, decentralization, and cryptographic verification. The study also identifies limitations and proposes directions for future research including scalability optimization through sharding and integration of the Internet of Things (IoT) for automated data capture.**

Keywords—blockchain; smart contracts; supply chain; product tracking; Ethereum; transparency; data integrity; traceability

I. INTRODUCTION

Supply chains represent one of the most complex operational ecosystems in modern industry, spanning manufacturers, logistics providers, distributors, and retailers across international boundaries. The integrity of transaction data across these networks is fundamental to operational efficiency, regulatory compliance, and consumer trust. However, the predominant approach of maintaining separate, centralized ledgers at each node of the supply chain introduces structural vulnerabilities that undermine these objectives [1].

Traditional ledger systems suffer from several well-documented deficiencies. Each supply chain participant maintains its own database, creating data silos that make cross-organizational verification cumbersome and expensive. Centralized databases are susceptible to both insider and outsider threats, enabling unauthorized modification of records. The reconciliation of disparate data sources during audits demands considerable time and financial resources. Furthermore, the absence of real-time visibility across the chain prevents rapid identification of bottlenecks, fraud, or quality failures.

Blockchain technology, originally conceptualized for cryptocurrency transactions, has emerged as a promising architectural foundation for addressing these challenges. A blockchain is a distributed, append only ledger maintained by a decentralized network of nodes, where records are grouped into cryptographically linked blocks and validated through consensus mechanisms [2]. Its inherent properties of immutability, transparency, and decentralization align directly with the data integrity requirements of supply chain management.

Smart contracts self-executing programs stored on the blockchain whose terms are directly written in code—extend the utility of blockchain from passive record-keeping to

active business logic enforcement [3]. By encoding product lifecycle rules within smart contracts, supply chain stakeholders can automate the creation, update, and retrieval of product records without reliance on a trusted central authority.

This paper proposes and analyzes a blockchain-based supply chain product tracking system implemented through Ethereum smart contracts. The system adopts a role-based design in which specific functions are restricted to registered participants. The contract enforces permissions through an on-chain role registry, requiring users to be assigned as Admin, Manufacturer, or Distributor before they can modify product data. The contribution of this work is threefold: (1) a coherent system architecture integrating a role-restricted smart contract with supply chain workflows; (2) detailed smart contract design for product lifecycle management; and (3) a rigorous security and transparency analysis grounded in established vulnerability and defense frameworks from the literature.

II. Related Works

A. Blockchain Architectures and Foundations

A comprehensive survey of blockchain theories, modelings, and tools by Xie et al. [2] identifies three fundamental architectural variants: public blockchains, private blockchains, and consortium blockchains. Public blockchains such as Ethereum permit permissionless participation, offering maximum decentralization but lower throughput. Private and consortium blockchains restrict network membership to verified participants, trading some decentralization for performance gains that may be desirable in enterprise supply chain contexts. The survey emphasizes that consensus mechanisms ranging from Proof of Work (PoW) to Proof of Authority (PoA) and Byzantine Fault Tolerance (BFT) variants are central architectural decisions that determine throughput, finality, and security guarantees.

Scalability remains a fundamental constraint of blockchain architectures. Hafid et al. [1] survey sharding as a horizontal scaling technique in which the blockchain network is partitioned into smaller subnetworks (shards), each processing a subset of transactions in parallel. The authors classify sharding schemes into network sharding, transaction sharding, and state sharding, and identify cross-shard transaction atomicity as the primary outstanding challenge. In the context of supply chain tracking, where transaction volume may be high during peak shipping periods, sharding-based scalability mechanisms represent an important architectural consideration for production deployments.

The security reference architecture proposed by Homoliak et al. [4] provides a structured model for studying blockchain vulnerabilities across multiple layers: network, consensus, smart contract, and application. This layered analysis is particularly useful for supply chain systems, as threats may originate at any layer from network-level Sybil attacks to application-level access control failures.

.

*B. **Smart Contract Platforms, Challenges, and Security***

Smart contracts were formally introduced by Szabo and operationalized at scale through the Ethereum platform, which provides the Solidity programming language and the Ethereum Virtual Machine (EVM) for contract execution. Zheng et al. [3] provide a comprehensive overview of smart contract challenges including code immutability (once deployed, contracts cannot be altered without redeployment), gas consumption optimization, and the oracle problem for off-chain data integration. These challenges are directly relevant to supply chain smart contract design, where product data may need to reference external systems such as IoT sensors or ERP databases.

He et al. [5] survey smart contract construction and execution paradigms across multiple platforms including Ethereum, Hyperledger Fabric, and EOS. The authors highlight that formal specification of contract behavior prior to deployment is critical for security, as post-deployment modification is costly and potentially disruptive. This observation motivates the use of formal verification tools such as Solidifier [6], a bounded model checker for Solidity that allows developers to specify and verify contract properties before deployment, reducing the risk of logic errors that could be exploited in supply chain contexts.

Maintenance-related concerns for post-deployed contracts are catalogued by Huang et al. [7], who identify bug fixes, gas optimization, and feature additions as the three primary categories of post-deployment maintenance need. The immutability of deployed contracts means that upgradeability must be explicitly designed into the system architecture. For example, through proxy patterns—adding complexity that must be weighed against security benefits.

*C. **Smart Contract Vulnerability Analysis***

A substantial body of literature has catalogued vulnerabilities in Ethereum smart contracts. The systematic review by Sayeed et al. [8] identifies reentrancy, integer overflow/underflow, timestamp dependence, and front-running as the most critically exploited vulnerability classes in deployed Ethereum contracts. Reentrancy, responsible for the infamous DAO hack of 2016, occurs when an external contract call is made before internal state updates are completed, allowing recursive calls to drain funds or corrupt state.

Comprehensive surveys by Atzei et al. [9], Chen et al. [10], and Liao et al. [11] converge on a taxonomy of vulnerabilities spanning EVM-level bugs (stack size limitations, gas cost

manipulation), Solidity-level flaws (visibility modifiers, uninitialized storage pointers), and application-level issues (access control failures, business logic errors). For supply chain contracts specifically, access control vulnerabilities are of paramount concern: unauthorized parties must be prevented from calling CreateProduct() or UpdateProduct() functions, as unauthorized writes would corrupt the chain's data integrity guarantees.

Detection and mitigation methodologies are surveyed by Liu et al. [12] and Vacca et al. [13], who categorize approaches into static analysis (examining source code or bytecode without execution), dynamic analysis (fuzzing and symbolic execution during runtime), and formal verification (mathematical proofs of contract correctness). Tools such as Mythril, Slither, Manticore, and Oyente represent the practical toolkit available to developers. The survey on security defense for smart contracts by Zhou et al. [14] further classifies defense strategies at the language level (safe coding patterns), tool level (automated vulnerability detectors), and protocol level (circuit breakers, multisig requirements), providing a defense-in-depth framework applicable to supply chain contract hardening.

Survey work on security verification of blockchain smart contracts by Zhang et al. [15] distinguishes between pre-deployment verification (formal methods, model checking, theorem proving) and post-deployment monitoring (on-chain anomaly detection, event logging). The authors argue that a combination of both is necessary for high-assurance applications, a conclusion that directly informs the security analysis in Section VI of this paper.

D. *Blockchain in Supply Chain and Finance*

The intersection of blockchain and supply chain management has attracted significant research attention. The Fabric-SCF system proposed by Xu et al. [16] applies Hyperledger Fabric to supply chain finance, implementing role-based access control through channel and chain code policies to restrict document visibility. While permissioned architectures offer stronger confidentiality guarantees, the present work takes a contrasting approach, a permissionless design where transparency and wallet attribution replace role enforcement, reducing deployment complexity at the cost of application-layer responsibility for access control.

Critical literature reviews on blockchain and supply chain finance by Gelsomino et al. [17] and Jiang et al. [18] analyze the intersection from operations, finance, and legal perspectives. Gelsomino et al. [17] identify transaction transparency, reduced reconciliation costs, and smart contract-automated payment triggers as the primary value propositions of blockchain in supply chain finance, while cautioning that regulatory uncertainty and integration complexity with legacy systems remain significant barriers to adoption. Jiang et al. [18] extend this analysis with a forward-looking taxonomy of blockchain supply chain finance models, distinguishing account receivable financing, inventory financing, and prepayment financing as distinct use cases with different blockchain implementation requirements.

The application of blockchain to medical information security by Zhang et al. [19] provides a relevant analogy to supply chain data protection. The authors implement a fine-grained access control scheme on Ethereum where data owners define access policies enforced by smart contracts. While the present system adopts a simpler permissionless model, the access policy pattern of Zhang et al. [19] represents a natural extension path for future versions requiring confidential data access controls.

III. Proposed System Architecture

A. *Architectural Overview*

The proposed system adopts a layered architecture consisting of four principal tiers: the User Layer, the Application Layer, the Smart Contract Layer, and the Blockchain Network Layer. Unlike generic permissionless models, this system implements an on-chain Role-Based Access Control (RBAC) mechanism. This Ensures only authorized participants (Admins, Manufacturers, and Distributors) can modify the state of the ledger, while also maintaining public transparency for tracking and auditing. This stratification follows established blockchain system design patterns identified in the literature [2][4] and is illustrated conceptually in the architectural description below.

The architecture is designed for deployment on a public or private Ethereum-compatible network. Because the contract imposes no role-based restrictions, any Ethereum wallet can interact with it directly, making deployment on a public testnet (e.g., Sepolia) or a private chain equally viable. Ethereum's Proof of Stake consensus provides deterministic finality while retaining full EVM smart contract programmability.

TABLE I. *System Architecture Layers*

Layer	**Table Column Head**	
	Components	***Function***
User Layer	Any Ethereum Wallet (Manufacturer, Distributor, Retailer, Auditor, Consumer)	Initiate transactions; query product data via web/mobile interface
Application Layer	Web DApp, REST API Gateway, Wallet Integration	User authentication, transaction signing, UI for product management

Layer	Table Column Head	
	Components	***Function***
Smart Contract Layer	ProductRegistry.sol (Solidity)	CreateProduct(), UpdateProduct(), getProuctHistoryCount(), getProductHistoryItem() logic enforcement
Blockchain Network Layer	Consortium Ethereum Nodes, Consensus Engine (PoA)	Transaction validation, block production, immutable state storage

B. System Participants

The SupplyChain contract is permissionless: no on-chain role registry exists, and no access modifiers restrict which wallet addresses may call the public functions. Any Ethereum account that holds a valid wallet may interact with the system. In practice, the following participant types are expected to use the system:

- Manufacturer: Registers new products on the blockchain by calling createProduct(). Although no on-chain restriction prevents other parties from calling this function, off-chain application-layer controls (e.g., DApp authentication) can be applied to limit product creation in practice.
- Admin: Responsible for system orchestration, including creating products and managing the roles of other participants.
- Distributor: Calls updateProduct() to record logistics eventspickup, transit, and delivery to the next nodeat each handoff point. Their wallet address is stored in the TrackingUpdate history entry, providing attribution without requiring prior authorization.
- Public: Any unassinged wallet could still query the product data and view the complete history in a read-only permision.
- **Retailer: Calls updateProduct() to record final delivery, and calls getProduct() or getProductHistoryItem() to retrieve full product provenance for inspection or consumer display.**
- **Auditor / Consumer: May call getProduct() and getProductHistoryItem() at any time to verify product** provenance without any on-chain permission requirement, since both are public view functions.

C. Data Storage Model

Product data is stored in the Ethereum state trie through Solidity mappings and structs. The on-chain data model stores product identifiers, metadata, and status information directly in contract storage, with only the content-addressed hash recorded on-chain. This hybrid storage model balances on-chain data integrity guarantees with the practical gas cost constraints of storing large data objects directly in contract storage.

Each product record maintains a history array that appends an immutable HistoryEntry struct each time UpdateProduct() is called. This append-only history structure, combined with Ethereum's immutable transaction log, provides complete and tamper-evident provenance from manufacture through final delivery.

IV. Smart Contract Design

A. Contract Overview

The SupplyChain smart contract is implemented in Solidity version ^0.8.20 and deployed on an Ethereum-compatible network. The contract manages the full product lifecycle through two principal data structures, a mapping-based storage model, and three public-facing functions. Solidity 0.8.x provides built-in arithmetic overflow and underflow protection [8], eliminating one of the most historically exploited vulnerability classes without requiring external SafeMath libraries. The complete contract source is presented and analyzed across the subsections below.

B. Data Structure

The contract defines two structs to manage product metadata and the history trail. Note the inclusion of logistical fields such as destination and quantitiy:

```
pragma solidity ^0.8.20;

contract SupplyChain {

  enum Role { None, Admin,
Manufacturer, Distributor }

  struct Product {
    string id
    string name
    string location
    string destination
    uint256 quantity
    string company
    string notes
    string status
    uint256 createdAt
    uint256 updatedAt
    bool exists
  }

  struct HistoryItem {
    uint256 time
    string location
    string status
```

```
    string desc
  }

  mapping(string => Product)
private products
  mapping(string => HistoryItem[])
private productHistory
  string[] private productIds
  mapping(address => Role) public
userRoles

  event UserRoleUpdated(address
user, Role role)
  event ProductCreated(string id,
string name, string location,
                       string
destination, uint256 quantity,
                       string
company, string status)
  event ProductUpdated(string id,
string newLocation,
                       string
newStatus, uint256 updatedAt)

  modifier onlyAdmin()
  modifier
onlyAdminOrManufacturer()
  modifier onlyAuthorizedUpdater()
  modifier
onlyExistingProduct(string _id)
}
```

The Product struct uses a bool exists sentinel field as the canonical existence check, enabling O(1) validation through the productExists modifier without iterating storage. The products mapping is declared public, granting automatic getter generation by the Solidity compiler. The productHistory mapping is private, enforcing that history retrieval is mediated exclusively through the contract's dedicated getter functions rather than direct external access. Two eventsProductCreated and ProductUpdatedare emitted on every state-changing call, supporting off-chain indexing via tools such as The Graph for efficient product search and dashboard updates.

C. createProduct() Function

The createProduct() function registers a new product on the blockchain. It accepts a caller-supplied product ID, name, and origin location, validates uniqueness, initializes the Product struct, appends the first TrackingUpdate to the history array, and emits the ProductCreated event:

```
function createProduct(
  string _id, string _name, string
_location,
  string _destination, uint256
_quantity,
  string _company, string _notes
) [onlyAdminOrManufacturer]:
  require _id, _name, _location,
_destination not empty
  require _quantity > 0
  require _company not empty
  require products[_id].exists ==
false  // no duplicate

  nowTs = block.timestamp
  initialStatus = "Menunggu"

  products[_id] = Product(
    id, name, location,
destination, quantity,
    company, notes,
status="Menunggu",
    createdAt=nowTs,
updatedAt=nowTs, exists=true
  )

  productIds.push(_id)

  initialDesc = "Produk
didaftarkan — {quantity} unit
                 menuju
{destination} ({company})"

  productHistory[_id].push(
    HistoryItem(time=nowTs,
location=_location,
                status="Menunggu",
desc=initialDesc)
  )

  emit ProductCreated(_id, _name,
_location, _destination,
                      _quantity,
_company, "Menunggu")
```

The uniqueness guard (require(!products[_id].exists)) prevents duplicate registration by leveraging the exists sentinel. The initial currentLocation is set equal to origin, reflecting that a newly created product has not yet been transported. The status field is initialized to the string literal "Created", establishing the starting state of the product lifecycle. Critically, the first TrackingUpdate entry is pushed to productHistory at creation time, ensuring that the history array is never empty for a valid product and that the creation event is itself part of the traceable record. The msg.sender field recorded in TrackingUpdate attributes the creation event to the calling account, providing on-chain non-repudiation of product registration.

D. updateProduct() Function

The updateProduct() function allows any caller holding a record of the product ID to submit a location and status update. It is guarded by the productExists modifier and appends an immutable TrackingUpdate entry to the history array:

```
function updateProduct(
  string _id, string _newLocation,
string _newStatus
) [onlyExistingProduct(_id),
onlyAuthorizedUpdater]:

  require _newLocation or
_newStatus not empty

  if _newStatus not empty:

_validateStatusForRole(userRoles[m
sg.sender], _newStatus)

  p = products[_id]
  nowTs = block.timestamp

  finalLocation = _newLocation if
not empty else p.location
  finalStatus   = _newStatus   if
not empty else p.status

  if both provided:
    desc = "Lokasi → {newLocation}
· Status → {newStatus} ({role})"
  else if only location:
    desc = "Lokasi → {newLocation}
({role})"
  else:
    desc = "Status → {newStatus}
({role})"

  p.location  = finalLocation
  p.status    = finalStatus
  p.updatedAt = nowTs

  productHistory[_id].push(
    HistoryItem(time=nowTs,
location=finalLocation,

status=finalStatus, desc=desc)
  )

  emit ProductUpdated(_id,
finalLocation, finalStatus, nowTs)
```

State mutations follow the checks-effects-interactions pattern: the productExists check is completed before any storage writes, and no external contract calls are made, eliminating reentrancy risk for this function [8][9]. The history array is append-only by construction—the EVM provides no native array-element deletion primitive that could silently remove entries—so the complete chronological record of all location and status changes is preserved immutably on-chain. Each TrackingUpdate records the msg.sender address, enabling attribution of every supply chain event to a specific participant wallet, which maps to a verified organizational identity through the network onboarding process.

E. getProduct() Function

The getProduct() function is a view function that retrieves current product state. As a view function it does not modify blockchain state and incurs no gas cost when called externally:

```
function getProduct(string _id)
  [view, onlyExistingProduct(_id)]
  → (id, name, location,
destination, quantity,
     company, notes, status,
createdAt, updatedAt):

  p = products[_id]
  return (p.id, p.name,
p.location, p.destination,
          p.quantity, p.company,
p.notes, p.status,
          p.createdAt,
p.updatedAt)
```

The function loads the Product struct into a memory copy (Product memory p) rather than accessing storage directly, which is a gas-efficient pattern for reading multiple fields from the same struct. The productExists modifier is applied before any data access, ensuring that queries for non-existent products revert cleanly. The function does not expose the internal exists flag in its return tuple, keeping the interface semantically clean for consumers.

F. History Retrival Function

Product provenance history is exposed through two complementary view functions. This two-function pattern—a count getter paired with an indexed item getter—avoids returning an unbounded dynamic array in a single call, which could cause out-of-gas failures for products with long histories [9][14]:

```
function
getProductHistoryCount(string _id)
  [view, onlyExistingProduct(_id)]
→ uint256:
  return
productHistory[_id].length

function
getProductHistoryItem(string _id,
uint256 _index)
  [view, onlyExistingProduct(_id)]
```

```
  → (time, location, status,
desc):
  require _index <
productHistory[_id].length
  h = productHistory[_id][_index]
  return (h.time, h.location,
h.status, h.desc)

function getAllProductIds() [view]
→ string[]:
  return productIds

function getProductCount() [view]
→ uint256:
  return productIds.length

function productExists(string _id)
[view] → bool:
  return products[_id].exists
```

The caller first invokes getProductHistoryCount() to determine the number of history entries, then iterates getProductHistoryItem() with indices from 0 to count-1 to retrieve the full provenance trail. Each returned entry includes the location, status, the Ethereum address of the account that submitted the update, and the block timestamp. The index bounds check (require(_index < productHistory[_id].length)) prevents array out-of-bounds access, which in older Solidity versions could cause undefined behavior [8]. The productHistory mapping is private, so these two functions are the sole sanctioned interface for history access, enabling the contract to enforce access policies in future upgrades without breaking external integrations.

V. System Workflow

A. Product Registration Workflow

The product registration workflow is initiated when a manufacturer prepares a new product batch for distribution. The workflow proceeds as follows:

Step 1: Wallet Connection & Role Verification: The manufacturer connects their Ethereum wallet (e.g., MetaMask) to the DApp. The system performs an on-chain check of the user's role. Access to the product registration interface is restricted to wallets assigned to the Manufacturer or Admin role.

Step 2: Product Data Entry: The manufacturer enters core product attributes, including the unique Product ID (string-based), product name, quantity, destination, and company name. These fields provide the primary tracking parameters for the shipment. Currently, metadata is stored directly on the blockchain ledger for maximum transparency.

Step 3: Transaction Signing: The DApp constructs a createProduct() transaction containing the product metadata. The wallet signs this transaction, and it is broadcast to the Ethereum Sepolia network.

Step 4: Network Validation: The Ethereum network nodes validate the transaction through the Proof of Stake (PoS) consensus mechanism on the Sepolia network. Once the transaction is included in a block, the contract state is updated, and the product becomes a permanent part of the supply chain's history.

Step 5: Event Emission: The smart contract emits a ProductCreated event. The frontend interface captures this event in real-time, allowing the newly created product to appear immediately on the dashboard and live tracking feed.

B. Product Update Workflow

The product registration workflow is initiated when a manufacturer prepares a new product batch for distribution. The workflow proceeds as follows:

Step 1: Wallet Connection: The manufacturer connects their Ethereum wallet (e.g., MetaMask) to the DApp. No on-chain role check is performed; any wallet may proceed.

Step 2: Product Data Entry: The manufacturer enters the product ID, product name, and initial location through the web interface. Optional metadata fields (batch number, production date, certification hash) are captured, and the updated location and status are entered into the tracking interface.

Step 3: Transaction Signing: The DApp constructs a createProduct() transaction, which the wallet signs using the connected private key. The signed transaction is broadcast to the network.

Step 4: Network Validation: The consortium nodes validate the transaction through the PoA consensus mechanism. Upon achieving consensus, the block is appended to the chain, and the contract state is updated.

Step 5: Event Indexing: The ProductCreated event is captured by the off-chain indexer, making the new product immediately discoverable through the DApp search interface.

C. Product Tracking Workflow

Any wallet participant, auditor, regulator, or consumer may invoke the tracking workflow at any time with no permission requirement:

Step 1 – Query Initiation: The user enters or scans a productId in the tracking interface.

Step 2 – Current State & History Retrieval: The DApp performs three distinct calls to the blockchain:

1. It calls getProduct() to retrieve the latest location and status.
2. It calls getProductHistoryCount() to find out how many times the product has been updated.
3. It iterates through getProductHistoryItem() for each index to rebuild the entire chronological timeline.

These are view functions, meaning they execute locally without incurring gas costs.

Step 3 – Data Rendering: The DApp renders a unified tracking dashboard. It displays the product's origin, destination, and current handler (company). Below the current state, it renders a Logistics Timeline, showing every past status update with its associated timestamp and a description of the role that performed the update.

Step 4 – Verification: The user may independently verify any entry by cross-referencing the displayed transaction hashes on a blockchain explorer, providing trustless auditability without reliance on the DApp itself.

VI. Security And Transparancy Analysis

A. Immutability and Tamper Resistance

Blockchain immutability constitutes the primary defense against data tampering, one of the most critical vulnerabilities in traditional supply chain record systems. Once a transaction is included in a block and sufficient blocks have been appended, the cryptographic chaining of block headers (each containing the hash of its predecessor) makes retroactive alteration computationally infeasible without controlling a majority of the network's computational resources [2][4].

In the proposed system, every createProduct() and updateProduct() call generates an on-chain transaction whose parameters—including the caller's wallet address, block timestamp, and new state—are permanently recorded on the blockchain. No account, including the contract deployer, can modify historical TrackingUpdate entries; the append-only productHistory array enforces this at the data structure level. This property directly addresses the data tampering problem identified in Section I and corroborated across the literature [8][9][11].

B. Decentralization and Trust Elimination

Traditional supply chains require trust in each individual organization's data management practices. The proposed system eliminates this requirement by distributing ledger maintenance across all network nodes. No central authority exists that can unilaterally alter the shared state, and every write is attributable to a specific wallet address recorded immutably in the TrackingUpdate history.

Because the contract is permissionless, the primary trust mechanism is transparency rather than access restriction: every write operation is publicly visible on-chain and permanently attributed to the calling wallet address. This wallet attribution model enables off-chain accountability, where organizations can be held responsible for the on-chain records their wallets produce. The absence of on-chain RBAC is a deliberate design trade-off that simplifies the contract and reduces attack surface at the access control layer [14], while acknowledging that application-layer controls remain necessary for production deployments.

C. Transparency and Real-Time Visibility

The combination of getProduct() and the productHistory array-getters provides all participants with identical, real-time visibility into the lifecycle of a product. By exposing the history through a count-and-item pattern (getProductHistoryCount and getProductHistoryItem), the system ensures that even products with extremely long histories can be audited without exceeding gas limits. Unlike traditional systems where data is siloed, this shared ledger presents a unified audit trail attributable to specific cryptographically authenticated wallets (Manufacturers and Distributors). Unlike traditional systems where each party sees only their portion of the supply chain, the shared blockchain ledger presents a unified view attributable to cryptographically authenticated sources. This addresses the lack of transparency identified in the problem background and aligns with the improved transparency outcome documented in blockchain supply chain finance implementations [16][17][18].

Event emission on CreateProduct() and UpdateProduct() calls enables additional transparency mechanisms including real-time dashboard updates and automated alerting when product status changes. These off-chain integrations, powered by event subscriptions, do not compromise the on-chain data integrity guarantees.

D. Smart Contract Vulnerability Mitigation

The contract design incorporates mitigations for the principal vulnerability classes identified in the literature. Table II summarizes the relevant vulnerabilities and corresponding mitigations:

TABLE II. *Smart Contract Vulnerability Mitigation Summary*

Vulnerability Class	Table Column Head	
	Risk in Supply Chain Context	**Mitigation Implemented**
Reentrancy [8][9]	Malicious contract calls UpdateProduct() recursively to inject false history entries	No Ether transfers in contract; checks-effects-interactions pattern applied; state updated before any external calls
Integer Overflow/Underflow [8]	productCount manipulation enabling ID collision	Solidity 0.8.x built-in overflow protection; explicit bounds checking on productId
Unrestricted Write Access [11][14]	Any wallet can call createProduct() or updateProduct(); no on-chain role restriction	Mitigated at application layer (DApp auth); on-chain: wallet attribution in every TrackingUpdate provides post-hoc accountability

Vulnerability Class	Table Column Head	
	Risk in Supply Chain Context	**Mitigation Implemented**
Timestamp Dependence [10]	Miner-manipulated timestamps affecting audit trail accuracy	Timestamps used for informational logging only; no business logic depends on block.timestamp for decisions
Denial of Service [9]	Large history arrays causing failures or timeouts during retrieval	Index-based history retrieval (getProductHistory Item); splits data into small, individual reads to ensure system stability even for long-lived products
Logic Errors [12][13]	Invalid status transitions corrupting lifecycle state machine	Status enum constraint; prohibition of Created reversion; formal property verification recommended via Solidifier [6]

E. Audit Efficiency

Audit processes in traditional supply chains require cross-organizational record reconciliation that can span weeks and involve significant professional fees. The proposed system transforms this process: a complete, tamper-evident audit trail for any product is accessible instantaneously through a sequence of getProduct() and getProductHistoryItem() calls. This provides a unified blockchain record maintained across all participants. Auditors may independently verify any entry by cross-referencing transaction hashes on a blockchain explorer, without depending on the cooperation of any supply chain participant.

The attribution of every state change to a specific blockchain address (which maps to a verified organizational identity through the onboarding process) provides non-repudiation of supply chain events, eliminating the disputes over record ownership and data provenance that characterize traditional audit processes. This aligns with the simplified auditing benefit documented in Fabric-SCF [16] and broader blockchain supply chain finance literature [17].

TABLE III. *BLOCKCHAIN-BASED VS. TRADITIONAL SUPPLY CHAIN TRACKING*

Parts	Table Column Head	
	Traditional System	**Proposed Blockchain System**
Data Integrity	Dependent on organizational controls; vulnerable to insider modification	Cryptographically enforced immutability; tamper-evident by design
Transaparency	Each party sees only its own records	Shared ledger; all authorized parties see unified history
Trust Model	Requires trust in each organization's data management	Trustless; cryptographic verification replaces organizational trust
Audit Process	Cross-organizational reconciliation; weeks/months; costly	Near-instant queries (getProduct); seconds; minimal cost
Acces Control	**Policy-based; human-enforced; error-prone; no public attribution**	**Permissionless; any wallet can write; accountability via wallet attribution on-chain**
Scalability	High; enterprise databases handle millions of TPS	Limited by blockchain throughput; mitigated by consortium/sharding
Implementation Cost	Lower initial cost; existing infrastructure	Higher initial deployment cost; consortium node infrastructure
Integration Complexity	Established EDI/API standards	Requires blockchain client integration; emerging standards

VII. DISCUSSION

A. Advantages of the Proposed System

The proposed blockchain-based product tracking system offers several substantive advantages over traditional centralized approaches. First, the immutability of the blockchain ledger provides a structurally superior data integrity guarantee compared to centralized databases that are vulnerable to both insider modification and external intrusion. The cryptographic linkage of blocks means that integrity can be verified by any participant without trusting a central authority.

Second, the smart contract layer replaces informal inter-organizational agreements with executable, automatically enforced business logic. The role-based acces design lowers the barrier to participation, any wallet can register and update products without prior enrollment while the event-driven

notification model provides real-time operational visibility that traditional EDI-based integration cannot match.

Third, the complete and attributable audit trail maintained in the product history array dramatically reduces the time, cost, and dispute risk associated with supply chain audits. Regulatory compliance reporting, product recall tracing, and dispute resolution are all facilitated by the single source of truth maintained on the blockchain.

B. Limitations and Challenges

Several limitations of the proposed approach merit acknowledgment. Scalability is a primary concern: the Ethereum main net's transaction throughput (approximately 15-30 transactions per second as of the PoW era, improved but still limited under PoS) may be insufficient for high-volume supply chains. While the consortium blockchain deployment with PoA consensus substantially mitigates this concern, very large supply chains may still require sharding solutions of the type surveyed by Hafid et al. [1].

The oracle problem represents a fundamental limitation for IoT-integrated deployments: the trustworthiness of on-chain records is only as strong as the trustworthiness of the off-chain data sources that generate CreateProduct() and UpdateProduct() transactions. If a participant's warehouse management system is compromised or its IoT sensors are spoofed, false data may be legitimately submitted to the blockchain. Blockchain can guarantee that the recorded data was not tampered with after recording, but cannot guarantee the accuracy of data at the point of recording.

Gas costs, while manageable in consortium deployments using PoA, remain a design consideration. The history array pattern, while providing complete provenance, will grow linearly with product lifecycle events, potentially increasing storage costs for long-lived products with many state transitions. Archival strategies such as compressing historical data to IPFS and storing only a Merkle root on-chain should be considered for products with particularly long or complex lifecycles.

Smart contract immutability, while a security benefit, also creates a maintenance challenge. Bug fixes and feature additions require new contract deployments and migration strategies. The proxy upgrade pattern addresses this but introduces complexity that itself becomes an attack surface if not implemented carefully [7]. Organizations must weigh the operational benefits of upgradability against the security risks of more complex contract architectures.

C. Comparison with Traditional Systems

Table III provides a structured comparison between the proposed blockchain-based system and traditional centralized supply chain tracking:

The template will number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first ...”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors’ names; do not use “et al.”. Papers that have not been published, even if they have been submitted for publication, should be cited as “unpublished” [4]. Papers that have been accepted for publication should be cited as “in press” [5]. Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation [6].

REFERENCES

[1] A. Hafid, A. S. Hafid, and M. Samih, "Scaling Blockchains: A Comprehensive Survey on Sharding," IEEE Access, vol. 8, pp. 125244–125262, 2020.

[2] R. Xie et al., "A Survey of State-of-the-Art on Blockchains: Theories, Modelings, and Tools," ACM Comput. Surv., vol. 54, no. 2, pp. 1–43, 2021.I. S. Jacobs and C. P. Bean, “Fine particles, thin films and exchange anisotropy,” in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[3] Z. Zheng, S. Xie, H. Dai, W. Chen, and H. Wang, "An Overview on Smart Contracts: Challenges, Advances and Platforms," Future Gener. Comput. Syst., vol. 105, pp. 475–491, 2020.R. Nicole, “Title of paper with only first word capitalized,” J. Name Stand. Abbrev., in press.

[4] I. Homoliak, S. Venugopalan, D. Reijsbergen, Q. Hum, R. Schumi, and P. Brejcha, "The Security Reference Architecture for Blockchains: Towards a Standardized Model for Studying Vulnerabilities, Threats, and Defenses," IEEE Commun. Surv. Tutor., vol. 23, no. 1, pp. 341–390, 2021.

[5] X. He, B. Qin, Y. Zhu, X. Chen, and Y. Liu, "A Comprehensive Survey on Smart Contract Construction and Execution: Paradigms, Tools, and Systems," Patterns, vol. 2, no. 2, p. 100179, 2021.K. Eves and J. Valasek, “Adaptive control for singularly perturbed systems examples,” Code Ocean, Aug. 2023. [Online]. Available: https://codeocean.com/capsule/4989235/tree

[6] A. Hajdu and D. Jovanovic, "Formalising and Verifying Smart Contracts with Solidifier: A Bounded Model Checker for Solidity," arXiv preprint arXiv:1907.09181, 2019.

[7] S T. Huang et al., "Maintenance-Related Concerns for Post-deployed Ethereum Smart Contract Development: Issues, Techniques, and Future Challenges," Empir. Softw. Eng., vol. 26, no. 4, p. 67, 2021.

[8] S. Sayeed, H. Marco-Gisbert, and T. Caira, "Systematic Review of Security Vulnerabilities in Ethereum Blockchain Smart Contracts," IEEE Access, vol. 8, pp. 216144–216162, 2020.

[9] W. Chen et al., "A Survey on Ethereum Systems Security: Vulnerabilities, Attacks and Defenses," ACM Comput. Surv., vol. 53, no. 3, pp. 1–43, 2021.

[10] P. Liao, T. Tang, C. Lin, and P. Wen, "Security Analysis Methods on Ethereum Smart Contract Vulnerabilities—A Survey," arXiv preprint arXiv:1908.08605, 2019.

[11] S. Chaliasos, B. Livshits, D. Sheridan, and A. Gervais, "Vulnerability Analysis of Smart Contracts," in Proc. ACM SIGSAC Conf. Comput. Commun. Secur., 2022.

[12] H. Liu et al., "Smart Contract Vulnerability Detection Technique: A Survey," in Proc. IEEE Int. Conf. Softw. Anal. Evol. Reengineering (SANER), 2021.

[13] A. Vacca, A. Di Sorbo, C. A. Visaggio, and G. Canfora, "A Systematic Literature Review of Blockchain and Smart Contract Development: Techniques, Tools, and Open Challenges," J. Syst. Softw., vol. 174, p. 110891, 2021.

[14] Y. Zhou et al., "Security Defense for Smart Contracts: A Comprehensive Survey," IEEE Trans. Dependable Secure Comput., 2023.

[15] L. Zhang et al., "A Survey on Security Verification of Blockchain Smart Contracts," IEEE Access, vol. 10, 2022.

[16] W. Xu, T. Zhao, J. Li, and H. Chen, "Fabric-SCF: A Blockchain-based Secure Storage and Access Control Scheme for Supply Chain Finance," IEEE Access, vol. 9, pp. 34438–34450, 2021.

[17] L. M. Gelsomino, M. Caniato, and R. Perego, "Blockchain and Supply Chain Finance: A Critical Literature Review at the Intersection of Operations, Finance and Law," Supply Chain Manag., vol. 26, no. 1, pp. 48–68, 2021.

[18] Y. Jiang et al., "A Survey on Blockchain-based Supply Chain Finance with Progress and Future Directions," Financ. Innov., vol. 8, no. 1, pp. 1–35, 2022.

[19] A. Zhang, X. Lin, "A Blockchain-based Secure Storage Scheme for Medical Information," IEEE/ACM Trans. Comput. Biol. Bioinform., 2021.

We suggest that you use a text box to insert a graphic (which is ideally a 300 dpi TIFF or EPS file, with all fonts embedded) because, in an MSW document, this method is somewhat more stable than directly inserting a picture.

To have non-visible rules on your frame, use the MSWord “Format” pull-down menu, select Text Box > Colors and Lines to choose No Fill and No Line.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove template text from your paper may result in your paper not being published.